graphice

PROJECT-BASED LEARNING

PHASE-II EVALUATION

Where's My Bus: A Hardware Independent Bus Tracker

Team Null Pointer — Team ID: DSCPP-III-2026-T007

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T007
Team Name: Null Pointer
Semester: 3rd
Sections: ML1, F, ML2, B
Domain: Tech.

Team Members

1. Devansh Dangwal (Team Lead) – 2510370109
2. Aryan Gusain – 2510012634
3. Vinayak Lakhera – 2510010715
4. Gautam Singh – 2510012593

Mentor

Mr.Kanishka Bhatt

Interactions completed: 4

Problem Statement & Finalized Objectives

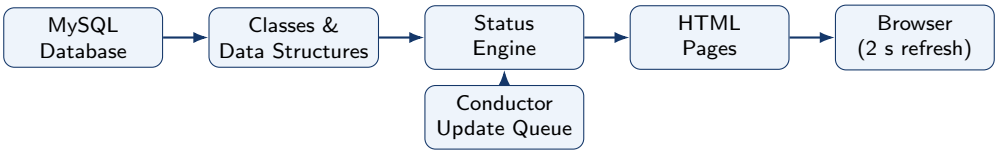
Problem Statement

Colleges with fixed bus routes give students no easy way to know whether their bus is late, full, or out of service. *Where's My Bus* gives every student a live colour status for each bus at their own stop, without GPS hardware or paid APIs.

Finalized Objectives

1. Show a colour-coded status for every bus at the student's chosen stop, with no login required.
2. Let the conductor update the bus position with one action at each stop.
3. Give the Head of Department (HOD) a summary report of all buses and recent updates.
4. Stay hardware independent and low cost: all routes, stops, buses and ETAs come from MySQL, built on hand-written data structures.

Updated System Architecture / Workflow



Major Components

- **Database:** stops, routes, order, buses, ETAs (MySQL)
- **Classes & data structures:** route = linked list; hash tables; queue and stack for updates; heap for sorting
- **Status engine:** moves bus along route, sets colour
- **Display:** C++ writes the HTML pages

Phase-II Design Decisions

- Each layer talks only to the next; nothing is hard-coded, all data loads from MySQL at start-up
- C++ runs the MySQL command-line client in the background and reads its output
- Colours are computed live and never stored
- Pages are written to a temp file, then moved, so the browser never sees a half-written page

Module Design & Functional Requirements

Major Modules

1. Database & loader (25 stops, 3 routes, 14 buses)
2. Core classes & data structures (Stop, Route, Bus, Update, users)
3. Status engine (movement, colours, undo/redo, trip completion)
4. Interfaces (terminal menu, student and HOD pages)

Functional Requirements

- Student views bus colour at a chosen stop without logging in
- Conductor logs in, starts a shift, marks next stop reached, marks seats full or maintenance, and can undo
- HOD gets a report of buses and recent updates
- Pages refresh every two seconds so changes appear immediately

Module Dependency

Interfaces → status engine → core classes → database (loaded at start-up). Conductor updates enter a queue, are validated, then go onto the history stack used for undo.

Technology Stack & Development Methodology

Technology Stack

- Programming: C++ (with STL)
- Database: MySQL 8
- Framework: none (plain HTML/CSS pages)
- Development Tool: g++ (Windows build file)
- Version Control: Git / GitHub

Development Methodology

1. Requirement Analysis
2. System Design
3. Module Development
4. Integration
5. Testing & Validation

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	Singly linked list (route as ordered stops), queue (conductor updates), stack (update history and undo/redo), binary heap (bus sorting), hash table (bus and route lookup)	Core classes, status engine
OOPs with C++	Classes, inheritance (User → Student / Conductor / HOD), class templates (Queue, Stack, Heap), STL, single-owner object management	User classes, registries, data structures
Operating Systems	Process creation: MySQL command-line client run from C++ as a background process, with its output read back	Database connection layer
DBMS	SQL schema for stops, routes, stop order, buses and ETAs; custom SQL query to get correct ETAs	Database, data loader

Development & Implementation Progress

Module	Status	Work Completed / Planned
Database	Completed	25 stops, 3 routes, 14 buses and ETAs between stops (can be extended)
C++–MySQL link & import	Completed	Connection through the MySQL client; all data loaded at start
Core classes & data structures	Completed	Stop, Route, Bus, Update, user classes, queue, stack, heap, hash tables
Status engine (drop trip)	Completed	Bus movement, colours, trip completion, undo and redo
Student & HOD pages	Completed	HTML/CSS pages with 2-second auto refresh (drop trip only)
Conductor terminal menu	Completed	Start shift, next stop, seats, maintenance, undo
Pickup, ETA, dashboard	Pending	Doubly linked list for return leg; ETA; web dashboard

Overall Project Completion: 70%

On schedule: drop-trip demo. Behind: pickup tracking, ETA, dashboard, manual, report.

Working Prototype / Demonstration

Implemented Features

- Live colour status of every bus at a selected stop (drop trips)
- Conductor menu: start shift, reach next stop, seat full, maintenance, undo
- HOD report with bus numbers and recent updates
- Student and HOD pages refresh automatically

Prototype / Output



Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Compilation check	No warnings	No warnings	Pass
Memory check	No leaks	No memory leak	Pass
Loading from MySQL	All routes and buses loaded	3 routes, 14 buses loaded	Pass
Conductor operations (drop trip)	All shift operations work	All performed successfully	Pass
Student page colours	Colours match bus state	Colours matched expected values	Pass
HOD report	Report shows completed trip	Shows the successful trip	Pass
Data structures	Heap order, list integrity, undo correct	All checks passed	Pass
Error messages	Clear error messages	Wrong password, missing config, bad database shown	Pass
Pickup tracking	Bus returns from rest stop to Graphic Era	Bus cannot return (forward-only list)	Fail

Results / Validation

8 of 9 test groups pass. The one failure, pickup tracking, comes from the forward-only route linked list and will be fixed with a doubly linked list.

GitHub Repository & Development Activity

Repository Information

- GitHub Link: `https://github.com/Nickyyayy/WheresMyBus`
- README.md: updated
- Branches: `main`, plus one branch per member
- Folders: `backend` (C++), `frontend` (CSS), `sql`, `output` (generated pages)
- Database password kept in a separate config file, not uploaded

Important Commits

- e9452e9 (HEAD, main): registries
- 4250780: data structures
- c012719: folder structure, Windows build file, SQL load and config, core classes
- a3cee68: MySQL database and README update

Individual commit counts: *no. of total commits/4*

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Devansh Dangwal	Team Lead, core logic	Stop, Route (linked list), Bus and Update classes; status engine with bus movement, colours, trip completion, undo and redo; slides and report	*25%*
Aryan Gusain	Interface & integration	Conductor/HOD terminal menu, configuration and main program; HTML/CSS pages; frontend–backend connection; slides and report	*25%*
Vinayak Lakhera	Database & data layer	MySQL database (25 stops, 3 routes, 14 buses, ETAs); C++–MySQL connection and import; User, Student, Conductor, HOD classes; hash tables; route validation	*25%*
Gautam Singh	Data structures & pages	Queue, Stack and Binary heap template classes and bus sorting; student and HOD HTML/CSS pages	*25%*

Challenges, Remaining Work & Phase-III Roadmap

Challenges & Solutions

- Graphic Era is first and last stop → track by stop order number, not name
- ETA view wrong for last record → own query
- C++–MySQL libraries failed on Windows → run the MySQL client from C++; quote paths with spaces
- Browser loaded half-written pages → temp file, then move
- Pointer/memory bugs → one owner per object, memory-checker tests
- One shared undo stack → pop other buses' newer updates, undo ours, push them back in order
- Pickup fails (forward-only route list) → doubly linked list planned

Phase-III Roadmap

1. Pickup tracking with a doubly linked list (Devansh, Vinayak)
2. Conductor web dashboard (Gautam, Aryan); ETA calculation (Devansh)
3. Larger database: more routes, stops and buses (Vinayak)
4. More tests: missed bus, resting bus, wrong updates; test on the demo laptop; code review (all)
5. User manual with screenshots, final report and final presentation

Expected Outcomes & References

Expected Final Outcomes

- Working tracker for both drop and pickup trips, with conductor web dashboard and ETAs
- Source code, SQL files, user manual, final report and demo
- Cost-effective college bus tracking with no GPS hardware or paid API
- Easily extended: more routes, stops and buses can be added through the database

References

1. MySQL 8.0 Reference Manual, dev.mysql.com/doc
2. C++ Standard Library reference, cppreference.com
3. Project repository: github.com/Nickyyayy/WheresMyBus

Thank You
Questions & Suggestions